浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：March 27, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　　指导老师：魏准　　实验类型：设计型实验
实验名称：Linear / Nonlinear SVM　　成　绩：　　签　名：

装 订 线

一、 SVM 线性分类

1. 实验目的

基于 Scikit-Learn 的支持向量机模块，使用鸢尾花数据集中的花瓣长度与宽度两个特征，完成线性 SVM 二分类（维吉尼亚鸢尾花 vs 其余类别）；重点测试花瓣长度 $\in [4, 6]$、花瓣宽度 $\in [0.8, 2.5]$ 监测区域内各点的分类情况，可视化决策超平面、间隔边界及支持向量，并对比不同 C 值的影响。

2. 实验原理

支持向量机（SVM）的目标是在特征空间中找到使两类样本间隔最大的超平面。对于线性可分情形，最优决策超平面满足：

$$\mathbf{w}^\top \mathbf{x} + b = 0, \quad \text{间隔宽度} = \frac{2}{\|\mathbf{w}\|}$$

为获得最大间隔，等价于求解以下凸优化问题：

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \ \forall i$$

软间隔 SVM 引入松弛变量 $\xi_i \geq 0$ 和正则化超参数 C，允许少量样本落在间隔内部：

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0$$

决策超平面由位于间隔边界上（或内部）的少数样本（支持向量）完全决定。

(1) Pipeline 工作流

实验代码使用 Scikit-Learn 的 `Pipeline` 将数据预处理与模型训练串联为一个整体，确保训练与预测阶段使用完全相同的变换参数：

$$\text{原始特征} \xrightarrow{\texttt{StandardScaler}} \text{标准化特征（零均值、单位方差）} \xrightarrow{\texttt{SVC(kernel="linear", C)}} \text{分类结果}$$

其中 `StandardScaler` 对花瓣长度和宽度进行标准化，避免量纲差异影响 SVM 的间隔计算；`SVC` 以线性核在标准化特征空间中求解最优超平面。

装 订 线

3. 核心代码实现

```python
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

#  加载数据集
iris = datasets.load_iris()
X = iris["data"][:, (2, 3)]            # 花瓣长度、花瓣宽度
y = (iris["target"] == 2).astype(int)  # 1: Virginica, 0: others

#  缩放特征 +  训练线性 SVM
svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="linear", C=1.0, random_state=42)),
])
svm_clf.fit(X, y)

#  监测区域：花瓣长度 4~6 cm，花瓣宽度 0.8~2.5 cm
pl_vals = np.arange(4.0, 6.1, 0.2)
pw_vals = np.arange(0.8, 2.6, 0.2)
pl_grid, pw_grid = np.meshgrid(pl_vals, pw_vals)
monitor_pts  = np.c_[pl_grid.ravel(), pw_grid.ravel()]
monitor_pred = svm_clf.predict(monitor_pts)

n_virginica = int(monitor_pred.sum())
n_other     = len(monitor_pred) - n_virginica
print(f"监测点总数: {len(monitor_pred)}")
print(f"判定为 Virginica    : {n_virginica}")
print(f"判定为 Non-Virginica: {n_other}")
```

Listing 1: SVM 线性分类核心代码（节选）

4. 运行结果

程序输出如下：

```
监测点总数       : 99
判定为 Virginica : 48 个
判定为 Non-Virginica: 51 个

 花瓣长度 花瓣宽度      预测类别   决策值
 -------- -------- ---------- --------
     ...      ...         ...      ...
     5.6      1.4 Non-Virginica -0.0126 <- 临界
     5.8      1.4    Virginica  0.2254 <- 越界
     4.0      2.0    Virginica  0.0126 <- 越界
     ...      ...         ...      ...

Training accuracy : 0.9533
Number of support vectors: [13 13] (class 0 / class 1)
Decision function (scaled): 2.0935*x1 + 2.4425*x2 + -2.8496 = 0
Margin width (scaled) : 0.6217
```

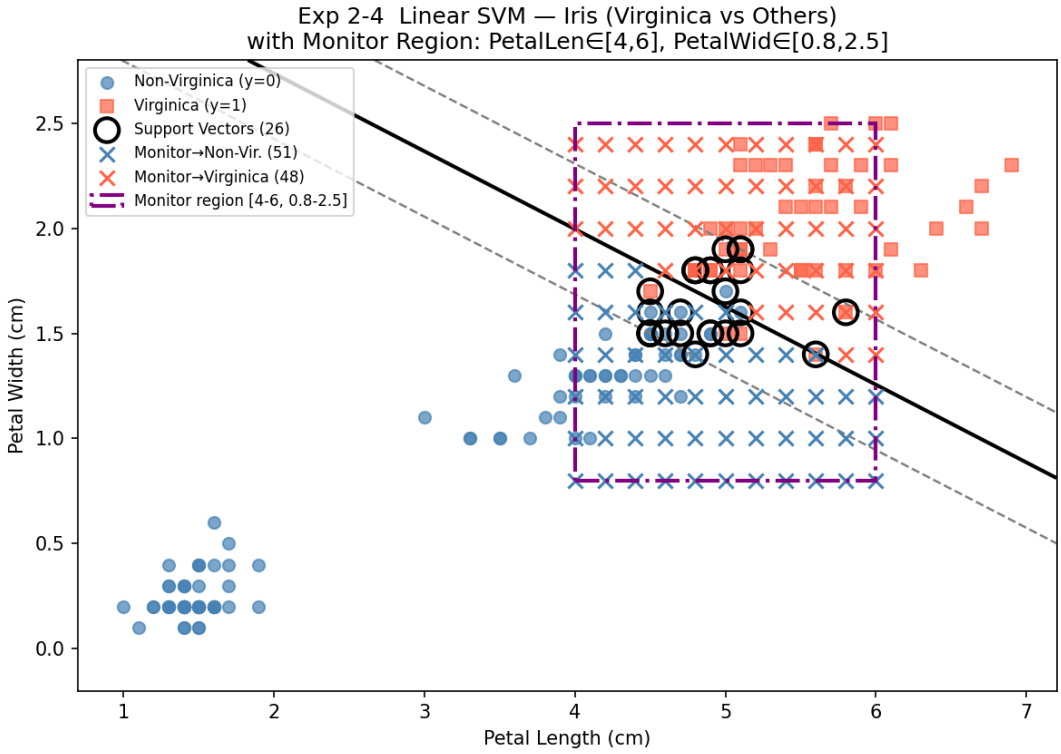


Figure 1: SVM 线性分类结果；紫色点划线矩形为监测区域 $[4,6]\times[0.8,2.5]$，× 号为监测点预测结果（蓝 × 为 Non-Virginica，红 × 为 Virginica）

5. 结果分析

如图 1 所示，线性 SVM 在标准化特征空间中找到了一条明确的分类超平面（黑色实线），并以虚线标出两侧等间隔边界。圆圈标注的 26 个支持向量（每类 13 个）完全决定了该超平面的位置与方向。训练集准确率为 0.9533，模型已将绝大多数维吉尼亚鸢尾花与其他类别正确区分。

对于监测区域（花瓣长度 $\in[4,6]$，花瓣宽度 $\in[0.8,2.5]$，共 99 个采样点）：

- **花瓣宽度** <1.4 **cm** 的所有点：决策值均为负，全部判定为 Non-Virginica；
- **花瓣宽度** ≥ 2.0 **cm** 的所有点：决策值均为正，全部判定为 Virginica；
- **花瓣宽度** $\in[1.4,1.8]$ **cm** 区域（约占 1/3）：为决策边界过渡带，分类结果同时取决于花瓣长度——花瓣越长、宽度越大越倾向于 Virginica。最终 99 个监测点中有 48 个判为 Virginica（48.5%），51 个判为 Non-Virginica（51.5%）。

这与决策超平面 $2.09\,x_1+2.44\,x_2-2.85=0$ 的数学形式一致：**花瓣宽度对分类的权重（2.44）略大于花瓣长度（2.09）**，说明在监测区域中花瓣宽度是更主要的判别依据。

6. 不同 C 值对分类结果的影响

(1) 实验设置

保持线性核不变，将正则化超参数 C 依次取 0.5、1、15、30、50、100，观察间隔宽度、支持向量数量及训练准确率的变化规律。核心代码如下：

```python
C_values = [0.5, 1, 15, 30, 50, 100]
for C in C_values:
    svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(kernel="linear", C=C, random_state=42)),
    ])
    svm_clf.fit(X, y)
    svc    = svm_clf.named_steps["svc"]
    acc    = svm_clf.score(X, y)
    margin = 2 / np.linalg.norm(svc.coef_[0])
    n_sv   = svc.n_support_
    print(f"C={C:6.1f}  acc={acc:.4f}  margin={margin:.4f}  n_sv={n_sv}")
```

Listing 2: SVM 线性分类——不同 C 值比较核心代码

(2) 运行结果

程序输出如下：

```
    C   Accuracy    Margin    n_sv (0/1)
--------------------------------------------------
  0.5     0.9533    0.6954   16 / 16
  1.0     0.9533    0.6217   13 / 13
 15.0     0.9667    0.3409    7 / 7
 30.0     0.9667    0.3288    7 / 7
 50.0     0.9600    0.2494    6 / 7
100.0     0.9600    0.2154    6 / 6
```

六种 C 值的决策边界可视化见图 2。

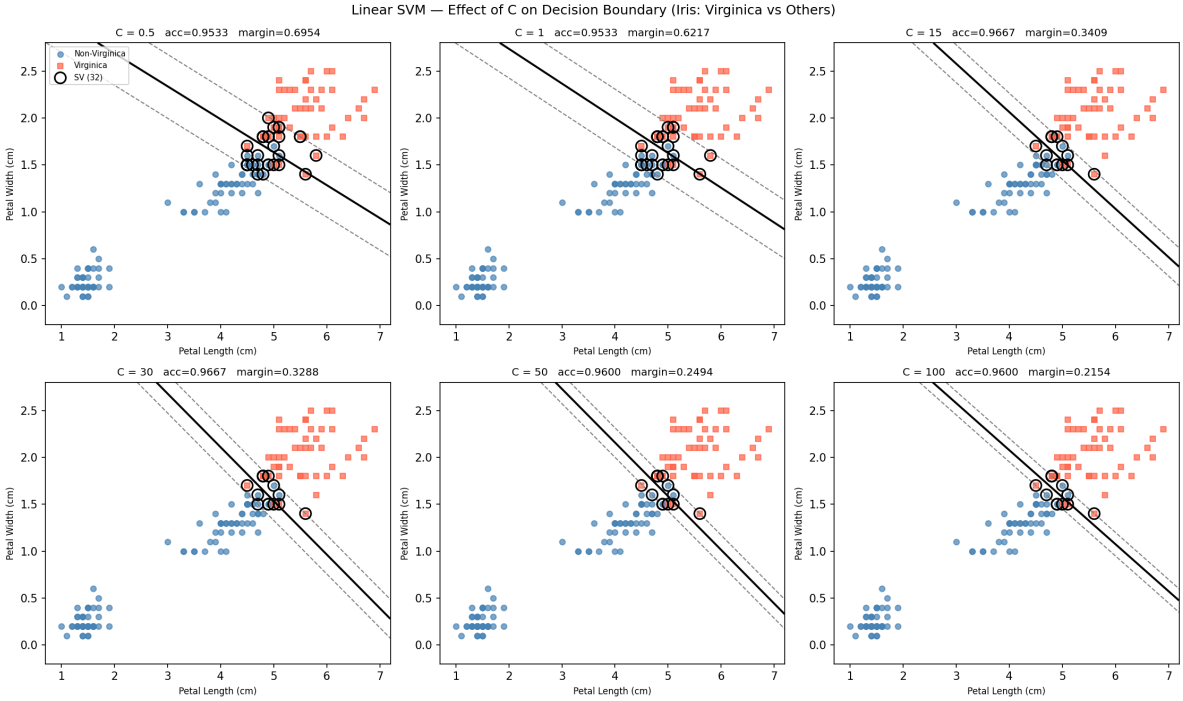


Figure 2: SVM 线性分类——不同 C 值下的决策边界（实线为决策面，虚线为间隔边界，圆圈为支持向量）

(3) 结果分析与结论

C 值与间隔宽度的关系 C 是软间隔 SVM 中惩罚误分类样本的正则化系数。

- C **较小（0.5、1）**：惩罚力度弱，模型优先追求更宽的间隔，允许更多样本落入间隔内甚至被误分类。间隔宽度分别为 0.6954 和 0.6217，支持向量数多（各 16 个、13 个），决策超平面位置由较多样本共同决定，方向更" 平均"。
- C **适中（15、30）**：惩罚力度增强，模型在减小训练误差与保持间隔之间取得平衡，准确率提升至 0.9667，间隔压缩至约 0.34，支持向量减少至 7 个/类。
- C **较大（50、100）**：模型进一步减少容忍的误分类样本，但准确率反而略降至 0.9600，间隔收窄至 0.25 和 0.22，支持向量最少（约 6 个/类）。这说明 C 过大时模型开始对噪声点过度拟合，泛化能力略有下降。

Table 1: 不同 C 值下的关键指标对比

C	训练准确率	间隔宽度（缩放空间）	支持向量数（0 类/1 类）
0.5	0.9533	0.6954	16 / 16
1	0.9533	0.6217	13 / 13
15	0.9667	0.3409	7 / 7
30	0.9667	0.3288	7 / 7
50	0.9600	0.2494	6 / 7
100	0.9600	0.2154	6 / 6

量化汇总

结论 C 值通过调节误分类惩罚力度，在**间隔宽度**（泛化能力）与**训练误差**（拟合能力）之间形成权衡：

(1) C 增大 $\Rightarrow$ 间隔变窄、支持向量减少、训练误差降低，但过大时有过拟合风险；

(2) C 减小 $\Rightarrow$ 间隔变宽、支持向量增多、模型更" 保守"，泛化能力通常更强；

(3) 本数据集上 $C \in [15, 30]$ 取得了准确率与间隔的最佳平衡（准确率 0.9667，间隔约 0.34）；C 继续增大至 50、100 后训练准确率不升反降，说明已出现轻微过拟合。

(4) 实践中应通过**交叉验证**（如 `GridSearchCV`）在保留验证集上选取最优 C，而非单纯依赖训练集准确率。

二、 SVM 非线性分类 I（多项式特征 + LinearSVC）

1. 实验目的

使用 Scikit-Learn 的线性分类器与多项式特征，对 `make_moons` 生成的月牙形非线性数据集完成二分类，具体目标如下：

(1) 产生非线性数据集（`make_moons`）并划分训练集与测试集；

(2) 利用 `Pipeline` 依次组合 `PolynomialFeatures`、`StandardScaler` 与 `LinearSVC`，构建非线性分类器；

(3) 可视化训练数据与测试数据；

(4) 讨论分析不同多项式阶数（2 阶、3 阶、8 阶）对分类结果的影响；

(5) 讨论分析不同正则化参数（$C = 1, 10, 100$）对分类结果的影响。

2. 实验原理

(1) 非线性 SVM 的基本思路

对于线性不可分数据集，一种直观的解决方案是将原始特征空间 $\mathbf{x} \in \mathbb{R}^p$ 通过显式的**多项式特征拓展**映射到高维空间 $\tilde{\mathbf{x}} \in \mathbb{R}^{p'}$，使得数据在新空间中线性可分，再用线性 SVM 进行分类：

$$\tilde{\mathbf{x}} = (X_1,\ X_1^2,\ X_2,\ X_2^2,\ \ldots,\ X_p,\ X_p^2,\ X_1X_2,\ \ldots)$$

超平面在原始特征空间中表现为非线性决策边界：

$$\beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \cdots + \beta_{2p-1} X_p + \beta_{2p} X_p^2 = 0$$

(2) 软间隔线性 SVM（LinearSVC）

软间隔 SVM 引入松弛变量 $\xi_i \geq 0$ 与惩罚系数 C，在拓展的特征空间上求解：

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{s.t.} \quad y_i(\mathbf{w}^\top \tilde{\mathbf{x}}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

其中 C 越大，对误分类的惩罚越重，决策边界越贴近训练样本（易过拟合）；C 越小，间隔越宽，泛化能力通常更强。`LinearSVC` 采用 hinge 损失函数，等价于上述软间隔优化问题的对偶形式。

(3) Pipeline 工作流

`Pipeline` 将数据预处理与模型训练串联为一个整体：

$$\text{原始特征} \xrightarrow{\text{PolynomialFeatures(degree)}} \text{高维特征} \xrightarrow{\text{StandardScaler}} \text{标准化} \xrightarrow{\text{LinearSVC(C)}} \text{分类结果}$$

标准化（零均值、单位方差）在多项式拓展后执行，避免高次项数值量级过大影响优化稳定性。

3. 核心代码实现

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# 1. 生成月牙形数据集并划分训练/测试集
X, y = make_moons(n_samples=200, noise=0.15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42   # 150 train / 50 test
)

# 2. 构建 Pipeline: PolynomialFeatures + StandardScaler + LinearSVC
polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=3)),   # 产生非线性特征
    ("scaler",        StandardScaler()),                 # 标准化
    ("svm_clf",       LinearSVC(C=10, loss="hinge",   # 线性 SVM (hinge 损失)
                                max_iter=5000, random_state=42)),
])

# 3. 训练与预测
polynomial_svm_clf.fit(X_train, y_train)
y_pred = polynomial_svm_clf.predict(X_test)

# 4. 评估
print(f"Train accuracy: {polynomial_svm_clf.score(X_train, y_train):.4f}")
print(f"Test  accuracy: {polynomial_svm_clf.score(X_test,  y_test ):.4f}")
print(classification_report(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))

# 5. 不同阶数对比（C=1 固定）
for deg in [2, 3, 8]:
    clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=deg)),
        ("scaler",        StandardScaler()),
        ("svm_clf",       LinearSVC(C=1, loss="hinge",
                                    max_iter=5000, random_state=42)),
    ])
    clf.fit(X_train, y_train)
    print(f"Degree={deg}  Train={clf.score(X_train,y_train):.4f}"
          f"  Test={clf.score(X_test,y_test):.4f}")

# 6. 不同 C 值对比（degree=3 固定）
for C in [1, 10, 100]:
    clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler",        StandardScaler()),
        ("svm_clf",       LinearSVC(C=C, loss="hinge",
                                    max_iter=5000, random_state=42)),
    ])
    clf.fit(X_train, y_train)
    print(f"C={C:<4}    Train={clf.score(X_train,y_train):.4f}"
          f"  Test={clf.score(X_test,y_test):.4f}")
```

Listing 3: SVM 非线性分类 I 核心代码

4. 数据可视化

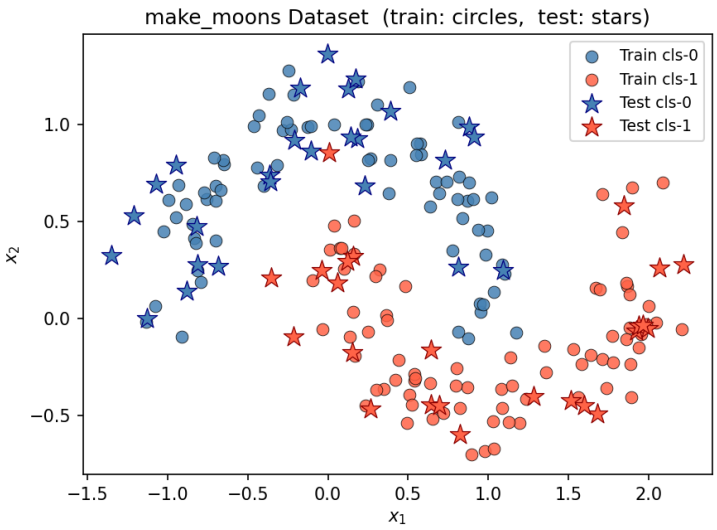


Figure 3: make_moons 数据集可视化：圆点为训练集（80 个样本），星号为测试集（20 个样本），蓝色为 Class 0，红色为 Class 1

如图 3 所示，月牙形数据集由两条相互缠绕的弧形点云组成，两类样本在原始二维特征空间中明显线性不可分，任何线性超平面均无法将其正确分开，需借助多项式特征拓展至高维空间。数据集共 200 个样本，按 3:1 划分训练集（150 个）和测试集（50 个），步长为 0.02，能够更真实地反映模型泛化能力。

5. 运行结果

(1) 控制台输出

```
训练集大小：150 测试集大小：50

【实验A】不同多项式阶数 (C=1)
 Degree  Train Acc  Test Acc
 ----------------------------------
      2     0.8533     0.8600
      3     0.9667     0.9800
      8     0.9800     0.9800

【实验B】不同 C 值 (degree=3)
      C  Train Acc  Test Acc
 ----------------------------------
      1     0.9667     0.9800
     10     1.0000     0.9800
    100     1.0000     0.9800

【最优模型详细报告】degree=3, C=10
              precision  recall f1-score support
           0       0.96    1.00     0.98      26
           1       1.00    0.96     0.98      24
    accuracy                         0.98      50
   macro avg       0.98    0.98     0.98      50
```

```
weighted avg    0.98      0.98      0.98       50

混淆矩阵:[[26  0]
 [ 1 23]]
```

(2)　不同多项式阶数的决策边界

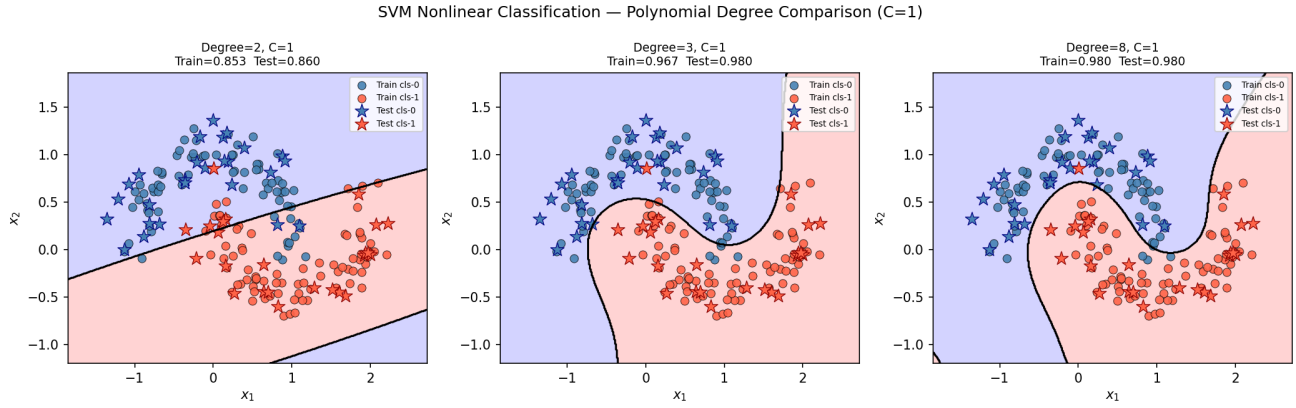


Figure 4: 不同多项式阶数（$C = 1$ 固定）的决策边界；圆点为训练集，星号为测试集；背景色为预测区域，黑线为决策边界

(3)　不同 C 值的决策边界

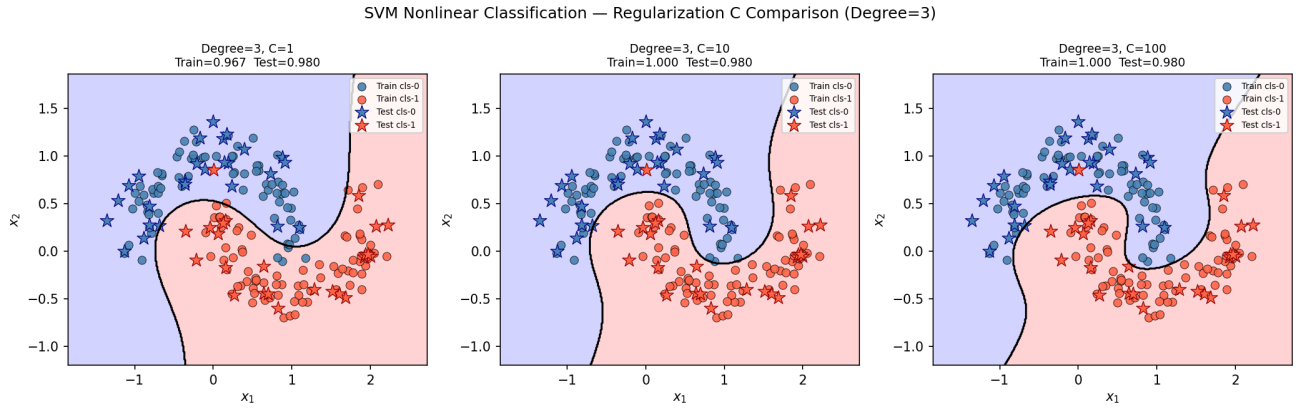


Figure 5: 不同正则化系数 C（Degree=3 固定）的决策边界；随 C 增大，决策边界更贴近训练点，过拟合风险升高

装 订 线

6. 结果分析

(1) 不同多项式阶数的影响

Table 2: 不同多项式阶数对比（$C = 1$ 固定，训练集 150 / 测试集 50）

阶数 (Degree)	特征维数	训练准确率	测试准确率
2	6	0.8533	0.8600
3	10	0.9667	**0.9800**
8	45	0.9800	**0.9800**

从表 2 和图 4 可以得出以下结论：

- **2 阶**：特征维数仅 6，模型复杂度最低。训练准确率 0.853，测试准确率 0.860，两者均为三种阶数中最低，表明 2 阶多项式边界过于简单，**欠拟合**明显——决策边界无法充分拟合月牙形曲线，仍有较多样本被误分类。
- **3 阶**：特征维数 10，训练与测试准确率达到 0.967 / 0.980，两者接近且均衡，是三者中综合最优的选择。决策边界足够弯曲以包络月牙形轮廓，同时未对噪声点过度敏感。
- **8 阶**：特征维数增至 45，训练准确率 0.980，测试准确率同为 0.980。虽然数值上与 3 阶相当，但由图可见决策边界已出现局部不规则波动（对部分噪声点过度拟合），模型的**过拟合风险**更高，且计算开销显著增大，并不优于 3 阶。

结论：多项式阶数控制模型复杂度。2 阶欠拟合，决策边界无法表达月牙形；3 阶在拟合能力与泛化之间取得最佳平衡；8 阶虽测试准确率与 3 阶持平，但边界更复杂、计算成本更高，实际中应优先选用 3 阶。

(2) 不同 C 值的影响

Table 3: 不同 C 值对比（Degree=3 固定，训练集 150 / 测试集 50）

C	训练准确率	测试准确率	特点
1	0.9667	**0.9800**	间隔较宽，容忍少量误分类
10	1.0000	**0.9800**	训练完全正确，间隔收紧
100	1.0000	**0.9800**	间隔更窄，过拟合风险升高

从表 3 和图 5 可以得出以下结论：

- $C = 1$：惩罚力度较弱，模型保持较宽间隔，训练准确率 0.967，测试准确率 0.980。在三个 C 值中，$C = 1$ 以最低的训练拟合度换得了与高 C 值相同的测试表现，模型最为” 保守” 且泛化稳定。
- $C = 10$：惩罚力度增大，训练准确率提升至 1.000（训练集完全正确），测试准确率维持 0.980，与 $C = 1$ 持平。间隔收紧，决策边界更贴近训练样本，但在此数据集上尚未出现泛化下降。
- $C = 100$：惩罚力度最强，训练准确率同为 1.000，测试准确率仍为 0.980，但由图可见决策边界局部出现更多不规则弯折，对噪声点的敏感程度进一步升高，**过拟合风险**最大，在更复杂或更嘈杂的数据集上泛化能力可能下降。

结论：在当前 Degree=3 设置下，三个 C 值的测试准确率均为 0.980，差异体现在训练准确率和边界复杂度上。$C = 1$ 以最宽间隔取得与更高 C 值相同的泛化效果，是本实验中最优的 C 选取；随 C 增大，模型逐渐从” 宽间隔容错” 转向” 零训练误差强拟合”，虽暂未导致测试准确率下降，但过拟合风险持续积累。

三、 实验总结

通过两组 SVM 实验，深入理解了支持向量机的核心原理与实践方法。

SVM 线性分类实验验证了线性 SVM 通过最大化间隔寻找最优决策超平面的思路，并通过对 $C \in \{0.5, 1, 15, 30, 50, 100\}$ 的系统比较，直观展示了软间隔参数 C 在容忍误分类与增大间隔之间的权衡——$C = 15 \sim 30$ 时准确率最高（0.9667），过大的 C（50、100）反而因过拟合导致准确率略降，说明选取合适的 C 对模型泛化至关重要。

SVM 非线性分类 I 实验通过多项式特征拓展（`PolynomialFeatures`）将线性不可分的月牙形数据映射到高维空间，再使用 `LinearSVC` 线性分类，验证了" 显式特征映射" 实现非线性分类的可行性。实验结果表明：

(1) **阶数**：2 阶明显欠拟合（训练/测试均约 0.86），3 阶与 8 阶测试准确率相当（均为 0.980），但 3 阶特征维数更低（10 维 vs 45 维）、边界更简洁，为最优选择；

(2) **C 值**：Degree=3 下，$C = 1, 10, 100$ 的测试准确率均为 0.980，但 $C = 1$ 以最宽间隔取得相同泛化效果，随 C 增大训练准确率达到 1.000 但过拟合风险持续积累，体现了偏差—方差权衡的核心规律；

(3) **实践意义**：多项式特征数随阶数指数增长（2 阶 6 维 $\rightarrow$ 8 阶 45 维），高阶时计算成本与过拟合风险同步上升，实际应用中应结合交叉验证选择合适阶数与 C 值。

整体而言，SVM 的核心超参数（C、特征阶数）对模型性能影响显著，实践中应结合交叉验证进行系统调优。SVM 算法理论完备（最大间隔与结构风险最小化），在小样本场景下具有较强的竞争力。

装 订 线